

Neural optimal feedback control with local learning rules

Friedrich

Optimal Feedback Control (OFC)

- Optimal Feedback Control (OFC) is a framework from control theory that describes how to compute actions (or controls) to achieve a goal in the most efficient way, especially in systems with uncertainty, like noise or delayed feedback.
 - OFC continuously adjusts actions based on real-time sensory feedback, using an internal model to predict future states and minimize a cost (like error or effort).

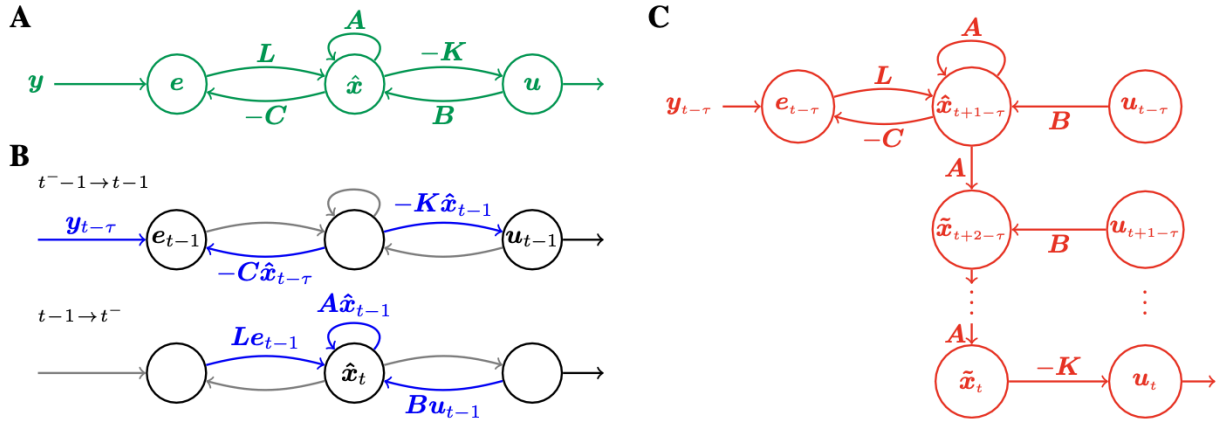


Figure S1: **Schematic of our proposed neural networks for OFC and alternative artificial neural network.** (A) Schematic of our proposed neural network (Bio-OFC). Nodes are annotated with the quantity they represent in their firing rates; edges are annotated with synaptic weights. (B) Input currents when updating e_{t-1} , u_{t-1} (top), and $\hat{x}_t$ (bottom). $\hat{x}_t$ is updated directly using the delayed measurement $y_{t-\tau}$, even for delays $\tau > 1$. (C) The alternative artificial neural network (ANN) that updates the past estimate $\hat{x}_{t+1-\tau}$, and predicts forward in time to estimate $\tilde{x}_{t+2-\tau}, \dots, \tilde{x}_t$, would require biologically implausible weight copies of A and B , as well as a memory of past controls

- $u_{t-\tau}, \dots, u_{t-1}$.

Kalman Filtering and Linear-Quadratic Regulator (LQR)

- Classical approaches like Kalman filtering and linear-quadratic regulators (LQR) work well in theory, but they're not biologically plausible.
- The **Kalman Filter (for Estimation)** is an algorithm used to **estimate** the hidden state of a system (like position or velocity) from noisy observations.
 - It combines predictions from a model with actual noisy observations to get the best estimate of the current state.
 - Example: You're tracking a moving object (like your hand). You don't see it clearly due to noisy visual input, but you can predict where it's going based on

how it was moving. The Kalman filter fuses that prediction and the noisy observation to guess where your hand really is.

- The **Linear-Quadratic Regulator (LQR) (for Control)** is a method to compute the optimal action (**control**) to apply to a system to minimize a cost, assuming the system's dynamics and noise are known and linear.
 - Given a current estimate of the state, it outputs a control action that minimizes a long-term cost function (usually quadratic—e.g., penalties on error and effort).
 - Example: It helps you smoothly move your hand to a target while using the least amount of energy and keeping the motion accurate.
- Why these aren't biologically plausible:
 - They require exact knowledge of system dynamics and noise statistics.
 - They often involve matrix operations, like inversion and solving Riccati equations—hard to do with real neurons.
 - They assume instant access to clean data (**no delays**) and global updates across the system.
 - Learning in these frameworks isn't local (i.e., neurons can't just use their own input/output to update weights).
- So, while Kalman filters and LQRs are mathematically elegant, the brain probably doesn't implement them directly, which is why papers like this one try to build **neural approximations that use biologically plausible rules**.

Core Idea of Bio-OFC

- The authors aim to model how the brain might solve control problems like moving your hand to a target despite sensory noise and delay, without needing to know the exact equations of motion or noise levels. Classical approaches like Kalman filtering and linear-quadratic regulators (LQR) work well in theory, but they're not biologically plausible.
- So they build a network that:
 - Uses adaptive Kalman filtering for estimating the system state from delayed/noisy observations.
 - Uses policy gradient (a reinforcement learning method) for learning a control policy, avoiding backpropagation and non-local updates.
 - Learns both the system dynamics (A, B, C) and Kalman gain (L) using online local learning rules, meaning updates are only based on information locally available at the synapse.

1. Bio-OFC Architecture:

- It contains neurons representing:
 - **State estimates** ($\hat{x}_t$),
 - **Prediction errors** (e_t),
 - **Control actions** (u_t).
- **Kalman filtering is modified** to work with delayed observations ($y_{t-\tau}$).

2. Learning Rules:

- **Local plasticity rules** update system parameters using Hebbian-like updates.
- Control weights are updated using **policy gradients with eligibility traces**, simulating dopamine-like global signals.

3. No Prior Knowledge Assumed:

- Doesn't assume knowledge of noise covariances or dynamics.
- Works **online** with **single-phase learning** (no alternating "learning" and "execution" phases).

4. Biological Plausibility:

- **No weight transport** or centralized computation.
- Supports **delayed feedback**, something prior models ignored.

5. Experiments:

- **Double integrator system** (a basic control problem),
- **Human hand-reaching task** (comparison with motor adaptation experiments),
- **2D winged flight** (tests performance in nonlinear and delayed scenarios).

Table 1: **Limitations of previously proposed neural implementations of OFC.** Presence or absence of different properties in previously proposed neural models, and their comparison to Bio-OFC. Guide to symbols: ✓: true, ✗: false, ✓✗: partially true, N/A: not applicable.

	[4]	[6]	[5]	[7]	[8]	Bio-OFC
delayed sensory feedback	✗	✗	✗	✗	✗	✓
control included	✗	✓	✗	✗	✗	✓
noise covariance agnostic	✗	✓	✗	✗	✗	✓
online system identification	✗	✓	✗	✓	✓✗	✓
local learning rules	N/A	✓	N/A	✗	✓	✓
tractable latent size	✗	✓	✗	✓	✓	✓
absence of inner loop	✓	✗	✓	✓	✗	✓
single phase learning/execution	N/A	✗	N/A	✓	✓	✓

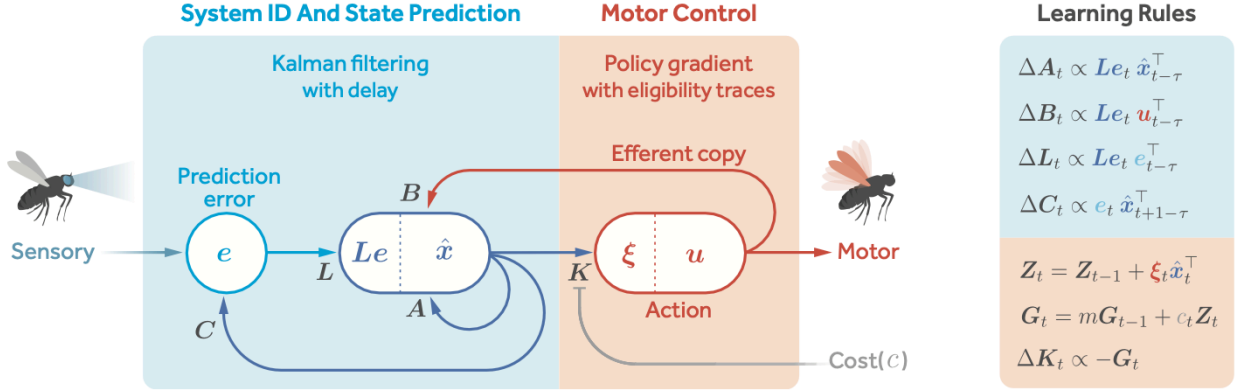


Figure 1: **The circuit and learning rules of the Bio-OFC algorithm.** Our circuit is comprised of two main parts. First (in blue), the circuit performs Kalman filtering. Then (in red), the circuit performs control using policy gradients with eligibility traces. Triangular arrowheads denote synaptic connections and the flat arrowhead denotes the modulatory effect of the cost signal.

Problem Formulation

2.1 Problem formulation

We model the environment as a linear dynamical system driven by control input and perturbed by Gaussian noise. The true state of the system $\mathbf{x}$ is hidden and all the animal has access to are the observations $\mathbf{y}$ that are assumed to be linear functions of the state corrupted by Gaussian noise.

$$\text{dynamics:} \quad \mathbf{x}_{t+1} = \mathbf{A}\mathbf{x}_t + \mathbf{B}\mathbf{u}_t + \mathbf{v}_t \quad (1)$$

$$\text{observation:} \quad \mathbf{y}_t = \mathbf{C}\mathbf{x}_t + \mathbf{w}_t \quad (2)$$

Here $\mathbf{v}_t \sim \mathcal{N}(0; \mathbf{V})$ and $\mathbf{w}_t \sim \mathcal{N}(0; \mathbf{W})$ are independent Gaussian random variables and the initial state has a Gaussian prior distribution $\mathbf{x}_0 \sim \mathcal{N}(\hat{\mathbf{x}}_0, \Sigma_0)$.

The goal is to estimate the latent state $\hat{\mathbf{x}}$ in order to design a control $\mathbf{u}$ that minimizes expected cost

$$\text{expected cost:} \quad J = \mathbb{E} \left[\sum_{t=0}^T c(\mathbf{x}_t, \mathbf{u}_t) \right] \quad (3)$$

$$\text{control:} \quad \mathbf{u}_t = k(\hat{\mathbf{x}}_t) = \arg \min J \quad (4)$$

where $c(\mathbf{x}_t, \mathbf{u}_t)$ is the instantaneous cost associated with state $\mathbf{x}_t$ and action $\mathbf{u}_t$. As the environment dynamics is not known to the animal a priori, the parameters $\mathbf{A}, \mathbf{B}, \mathbf{C}$ must be learned online.

1. Dynamics of the system (Eq. 1):

$$x_{t+1} = Ax_t + Bu_t + v_t$$

- x_t : the **hidden state** at time t (e.g., position and velocity of a limb).
 - u_t : the **control input** at time t (e.g., muscle activation).
 - A : state transition matrix (how state evolves on its own).
 - B : control matrix (how control input affects the state).
 - v_t : **process noise**, drawn from a Gaussian distribution with covariance V .
- This equation describes how the system evolves over time under control and noise.

2. Observation model (Eq. 2):

$$y_t = Cx_t + w_t$$

- y_t : **observed signal**, like sensory feedback.
 - C : observation matrix (how the hidden state is projected into observations).
- w_t : **observation noise**, also Gaussian with covariance W .
 - You don't see the true state x_t ; instead, you observe a noisy version of it, y_t .

3. Objective:

Estimate the **hidden state** $\hat{x}_t$ (using something like Kalman filtering), and use it to generate a control input u_t that minimizes the **expected cumulative cost**:

$$J = \mathbb{E} \left[\sum_{t=0}^T c(x_t, u_t) \right]$$

- $c(x_t, u_t)$: cost function — penalizes being far from the goal, or using too much control (like energy).
 - Goal: choose $u_t = k(\hat{x}_t)$ to minimize this cost over time.
- Challenge: You don't know the system parameters A , B , and C . The system must learn them online from data — just like a brain learning how to move in a new environment.
 - Recap

- A : State transition matrix — tells you how the current state x_t evolves into the next state x_{t+1} **without any control input**.
 | Example: if you're coasting on a bike, A predicts where you'll go just based on your momentum.
- B : Control matrix — tells you how your **actions** u_t (like pushing pedals) influence the next state.
 | Larger B means more effect from your inputs.
- C : Observation matrix — maps the true state x_t to the **observable signals** y_t .
 | Think of it like a camera or a sensor that only shows a noisy version of what's really going on.

- **Online Learning**

- “Online” learning means the system updates its knowledge step-by-step as data comes in — it doesn't wait for all the data to be collected.
- The system does not know how the world works ahead of time (i.e., it doesn't know A , B , C), so it has to infer them while interacting with the environment.
- In classical control, you assume A, B, C are known — you can compute everything ahead of time.
- But in the brain (or in real adaptive systems), you have to figure them out on the fly based on:
 - What you did (u_t)
 - What you saw (y_t)
 - And your estimate of what the system's state was ($\hat{x}_t$)
- This is also called system identification — learning the internal model of how actions and the environment evolve.
- In this paper, the authors develop local learning rules (i.e. biologically plausible updates) that allow a neural network to gradually estimate these matrices just using its **local activity** — which is pretty cool.

- **Local Activity**

- By local activity, we mean information that's available at a single neuron and its connections — specifically:
 - The neuron's own activity (e.g. firing rate),
 - Activity of its presynaptic (input) and postsynaptic (output) partners,
 - Possibly a global modulatory signal (like dopamine).
- In short: no need to access other parts of the network or perform matrix-wide computations. That's what makes a learning rule biologically plausible.

Kalman Estimation and Control

2.2 Kalman estimation and control

The Kalman filter [16, 2] is an estimator of the latent state $\hat{x}_t$ (and its variance) via a weighted summation of the current observation and the prediction of the internal model based on prior measurements. However, in biologically realistic situations the sensory feedback is always delayed. This means that the control signal u_t has to be issued, and thus the state x_t estimated, before y_t has been observed. The appropriately modified Kalman filter computes the posterior probability distribution of x_t given observations $y_{t-\tau}, \dots, y_0$ where $\tau \geq 1$ is the delay. We start here with the case of $\tau = 1$ for which the recursive updates of the mean $\hat{x}_t$ and the variance Σ_t are

$$\hat{x}_{t+1} = A\hat{x}_t + Bu_t + L_t(y_t - C\hat{x}_t) \quad (5)$$

$$\Sigma_{t+1} = (A - L_t C)\Sigma_t A^\top + V \quad (6)$$

where L_t is known as the Kalman gain matrix which optimally combines the noisy observations y_t with the internal model and is given by

$$L_t = A\Sigma_t C^\top (C\Sigma_t C^\top + W)^{-1}. \quad (7)$$

The Kalman filter is optimal in the sense that it minimizes the mean-squared error $\mathbb{E}[e_t^\top e_t]$ with prediction error (innovation) $e_t = y_t - C\hat{x}_t$.

The output feedback law, also known as policy, (4) simplifies if the cost J is quadratic in u_t and x_t :

$$u_t = -K\hat{x}_t \quad (8)$$

and is known as linear-quadratic regulator (LQR). The control gain, K , is found by solving a matrix Riccati equation (cf. Supplementary Material). Linear policies have been successfully applied to a variety of control tasks [17].

- **Estimate Part**

(5) Update of the state estimate (mean):

$$\hat{x}_{t+1} = A\hat{x}_t + Bu_t + L_t(y_t - C\hat{x}_t)$$

Let's unpack this:

- $A\hat{x}_t + Bu_t$: your **prediction** for what the next state should be.
- $y_t - C\hat{x}_t$: the **prediction error** (aka "innovation") — how much your prediction differs from what was observed.
- L_t : the **Kalman gain** — it decides how much to trust the observation vs. your internal model.

(6) Update of the estimate variance (uncertainty):

$$\Sigma_{t+1} = (A - L_t C)\Sigma_t A^\top + V$$

- This tracks your **uncertainty** about the state. As you get new observations, this should ideally go down.

(7) The Kalman gain formula:

$$L_t = A\Sigma_t C^\top (C\Sigma_t C^\top + W)^{-1}$$

This formula decides how to **weight your prediction error** when updating your belief. It adapts based on:

- Your confidence in your current state estimate (Σ_t),
- And how noisy the observations are (W).
- Kalman filtering is optimal in the sense that it minimizes the mean squared prediction error, $e_t = y_t - C\hat{x}_t$

• **Control Part**

Linear-Quadratic Regulator (LQR) is a classic method in control theory for computing the **optimal way to act** (i.e., choose control inputs u_t) in a linear system so that you minimize a given **cost function** over time.

It's called:

- **Linear**: because the system dynamics are linear (as in $x_{t+1} = Ax_t + Bu_t + \text{noise}$),
- **Quadratic**: because the cost function is quadratic in both the state and control.

Quadratic cost functions are used because they:

1. **Penalize deviation** from the desired state (like being far from a goal),
2. **Penalize excessive control effort** (e.g., you don't want to use too much energy).

A typical form of the cost is:

$$J = \sum_{t=0}^T (x_t^\top Q x_t + u_t^\top R u_t)$$

- $x_t^\top Q x_t$: penalizes how far the state is from zero (or a goal), scaled by matrix Q ,
- $u_t^\top R u_t$: penalizes large control inputs, scaled by R .
- Quadratic terms are nice because they're:
 - Smooth and convex $\rightarrow$ easy to optimize,
 - Closed-form solutions exist (e.g., for computing K).

(8) Control policy:

$$u_t = -K\hat{x}_t$$

- K is the **control gain** — tells you how strongly to respond to the estimated state.
- This is known as a **Linear Quadratic Regulator (LQR)**.
- More details:

- K is the **control gain matrix**.
- It tells you **how strongly to react to your current estimated state**.
- It is computed based on the system dynamics and the cost matrices Q, R by solving the **Riccati equation** (a specific matrix equation).

In practice:

- If K is large, the controller reacts **aggressively** to deviations (big corrections),
- If K is small, it's more **cautious** (gentle corrections).

○

A Linear-quadratic regulator (LQR)

The optimal control problem is to determine an output feedback law that minimizes the expected value of a cost criterion. If the cost J is quadratic, the optimal output feedback is a linear control law known as linear-quadratic regulator (LQR).

$$J = \mathbf{x}_T^\top \mathbf{Q} \mathbf{x}_T + \sum_{t=0}^{T-1} (\mathbf{x}_t^\top \mathbf{Q} \mathbf{x}_t + \mathbf{u}_t^\top \mathbf{R} \mathbf{u}_t) \quad (\text{S1})$$

$$\mathbf{u}_t = -\mathbf{K}_t \mathbf{x}_t \quad \text{with control gain} \quad \mathbf{K}_t = (\mathbf{B}^\top \mathbf{P}_{t+1} \mathbf{B} + \mathbf{R})^{-1} \mathbf{B}^\top \mathbf{P}_{t+1} \mathbf{A} \quad (\text{S2})$$

where $\mathbf{P}_t$ is determined by the dynamic Riccati equation that runs backwards in time

$$\mathbf{P}_{t-1} = \mathbf{A}^\top \mathbf{P}_t \mathbf{A} - (\mathbf{A}^\top \mathbf{P}_t \mathbf{B}) (\mathbf{R} + \mathbf{B}^\top \mathbf{P}_t \mathbf{B})^{-1} (\mathbf{B}^\top \mathbf{P}_t \mathbf{A}) + \mathbf{Q} \quad (\text{S3})$$

from terminal condition $\mathbf{P}_T = \mathbf{Q}$. Linear-quadratic-Gaussian (LQG) control uses the Kalman estimate $\hat{\mathbf{x}}$ in the controller, $\mathbf{u}_t = -\mathbf{K}_t \hat{\mathbf{x}}_t$.

○

- What does LQR give you?
 - A **feedback controller**: it maps your state estimate $\hat{\mathbf{x}}_t$ directly to the control input $\mathbf{u}_t$,
 - It's **optimal**: minimizes the total cost over time,
 - And it's **efficient**: once K is computed, control is just a simple matrix multiplication at every step.

○

• Linear-Quadratic-Gaussian (LQG) Control

This section connects **estimation (Kalman filter)** with **action (LQR)**:

1. Kalman filter gives you the best estimate of what's happening.
2. LQR tells you the best way to act on it.

Together, this is called **Linear-Quadratic-Gaussian (LQG) control** — powerful, but not biologically realistic unless you find ways to approximate it (which is what this paper is trying to do).

○

Neural Network Representation for Optimal Feedback Control

Inference:

3 Neural network representation for optimal feedback control

3.1 Inference

For now, let us assume that the system dynamics, the Kalman gain and the control gain A , B , $-C$, L , and K are constant and known. Then, the latent state $\hat{x}$ can be obtained by the Kalman estimator Eq. (5). This algorithm naturally maps onto the network in Fig. 1 and Supplementary Fig. S1A with neural populations representing $\hat{x}$, $e := y - C\hat{x}$, and u that are connected by synapses whose weights represent the elements of the matrices A , B , $-C$ and L . The computation of the control variable u according to Eq. (S2) can be implemented by synapses whose weights represent the elements of the matrix $-K$.

If the sensory stimulus delay is $\tau = 1$, our network implements the Kalman prediction reviewed in the previous section. In case $\tau > 1$, the latent state must be recomputed throughout the delay period which requires a biologically implausible circuit (cf. Supplementary Fig. S1C). To overcome this, we adapt an alternative solution from online control [18, 19]. Specifically, we combine delayed measurements with the similarly delayed latent state estimation. Such inference can be performed by the network of the same architecture and adjusting the synaptic delay associated with matrix C to match with the sensory delay, in accordance with the following expression:

$$\hat{x}_{t+1} = A\hat{x}_t + Bu_t + L \underbrace{(y_{t+1-\tau} - C\hat{x}_{t+1-\tau})}_{e_t} \quad (9)$$

As we show in Results (Fig. 3) this reduces predictive performance only modestly compared to the biologically unrealistic scheme. More details on the inference process and the temporal order in which our recurrent network performs the above steps is shown in Supplementary Fig. S1B.

-
- If $\tau = 1$,
 - use the equation from the previous section
- If $\tau > 1$,

When the sensory observation is **delayed by** $\tau > 1$, it's biologically unrealistic to **recompute all intermediate predictions** — that would require copying weights and holding long memory (not plausible for the brain).

So the authors propose a **workaround**:

- Instead of updating $\hat{x}_{t+1}$ with the current y_t ,
- They use the **delayed** observation $y_{t+1-\tau}$ and the **equally delayed prediction** $C\hat{x}_{t+1-\tau}$.

This gives the modified inference equation (Eq. 9):

$$\hat{x}_{t+1} = A\hat{x}_t + Bu_t + L(y_{t+1-\tau} - C\hat{x}_{t+1-\tau})$$

- The expression in parentheses is still the **prediction error** e_t , just shifted to account for delay.
- This approach:
 - Preserves biological plausibility — doesn't require unrealistic memory or backtracking.
 - Still works well, even with delay (as shown in their results).
 - Can be implemented using the same neural architecture, just by adjusting synaptic delay timing.
- So in short: this section shows how to make the Kalman estimation step biologically realistic, even when the brain gets delayed feedback, which is almost always the case (e.g., it takes time for your eyes or skin to send signals).

Learning: System Identification and Kalman Gain

3.2 Learning

3.2.1 System identification and Kalman gain

Next, we turn our attention to the system identification/learning problem, which was not addressed by some of the previous proposals, cf. Table 1. Given a sequence of observations $\{y_0, \dots, y_T\}$, we use a least squares approach [20] to find the parameters that minimize the mean-square prediction error $\frac{1}{T} \sum_{t=0}^T e_t^\top e_t$. We perform this optimization in an online manner, that is at each time-step $t + 1$, after making the delayed observation $y_{t+1-\tau}$, we update the parameter estimates $\hat{A}, \hat{B}, \hat{C}$ using steps that minimize $e_t^\top e_t$, assuming that the state estimate and actions corresponding to prior observations (e.g. $\hat{x}_{t-\tau}, u_{t-\tau}$ etc.) are fixed. To obtain L we would like to avoid solving the Riccati equation (6) as it requires matrix operations difficult to implement in biology. So, we use the same optimization procedure to update the Kalman gain L . Using Eq. (9) and explicitly writing out the matrix/vector indices as superscripts yields the following stochastic gradient with respect to A :

$$\begin{aligned} -\frac{\partial}{\partial A^{ij}} \frac{1}{2} \sum_k (e_t^k)^2 &= -\sum_k e_t^k \frac{\partial e_t^k}{\partial A^{ij}} = -\sum_k e_t^k \frac{\partial (y_{t+1-\tau}^k - \sum_l C^{kl} \hat{x}_{t+1-\tau}^l)}{\partial A^{ij}} = \\ &= \sum_{k,l} e_t^k C^{kl} \frac{\partial (\sum_m A^{lm} \hat{x}_{t-\tau}^m + \sum_n B^{ln} u_t^n + \sum_p L^{lp} e_t^p)}{\partial A^{ij}} = \\ &= \sum_{k,l,m} e_t^k C^{kl} \delta^{li} \delta^{mj} \hat{x}_{t-\tau}^m = \sum_k e_t^k C^{ki} \hat{x}_{t-\tau}^j \end{aligned} \quad (10)$$

Performing similar derivations for the other synaptic weights, our optimization procedure would rely on the following stochastic gradients:

$$-\nabla_A \frac{1}{2} e_t^\top e_t = C^\top e_t \hat{x}_{t-\tau}^\top \quad -\nabla_B \frac{1}{2} e_t^\top e_t = C^\top e_t u_{t-\tau}^\top \quad (11)$$

$$-\nabla_L \frac{1}{2} e_t^\top e_t = C^\top e_t e_{t-\tau}^\top \quad -\nabla_C \frac{1}{2} e_t^\top e_t = e_t \hat{x}_{t+1-\tau}^\top \quad (12)$$

This yields a classical Hebbian rule between (a memory trace of) presynaptic activity $\hat{x}_{t+1-\tau}$ and postsynaptic activity e_t for weights C . However, it suggests non-local learning rules for A, B, L , which runs contrary to biological requirements. We can circumvent this problem by replacing $C^\top$ with L , which corresponds to left-multiplication of the gradients with a positive definite matrix (see Supplementary Material Sec. D). This still decreases the mean-square prediction error under some mild initialization constraints on C and L and yields local plasticity rules, cf. Fig. 1.

$$\Delta \hat{A}_t \propto L e_t \hat{x}_{t-\tau}^\top \quad (13)$$

$$\Delta L_t \propto L e_t e_{t-\tau}^\top \quad (14)$$

$$\Delta \hat{B}_t \propto L e_t u_{t-\tau}^\top \quad (15)$$

$$\Delta \hat{C}_t \propto e_t \hat{x}_{t+1-\tau}^\top \quad (16)$$

where the input current $L e_t$ is locally available at neurons representing $\hat{x}$. The first three rules are local, but non-Hebbian, capturing correlations between presynaptic activity $\hat{x}_{t-\tau}, u_{t-\tau}, e_{t-\tau}$ and postsynaptic current $L e_t$. Note that these updates do not require knowledge of the noise covariances V and W , an advantage over previous work, cf. Table 1.

The goal is **system identification**: learning the unknown parameters of the model (A, B, C, L) **online**, i.e., while the system is running — like a brain learning its own dynamics through experience.

The authors use a **least squares approach**: they try to **minimize the squared prediction error**:

$$e_t = y_{t+1-\tau} - C\hat{x}_{t+1-\tau}$$

The idea is: the more accurate your prediction $\hat{x}_{t+1-\tau}$, the smaller e_t is.

So they minimize:

$$\frac{1}{2}e_t^\top e_t$$

•

They derive **gradients** of the squared error with respect to each matrix. For example:

$$\frac{\partial}{\partial A^{ij}} \left(\frac{1}{2}e_t^\top e_t \right)$$

They work out this derivative in a fully expanded matrix form in equation (10). Without diving into the math too deeply, the final result is:

- For A :

$$\Delta \hat{A}_t \propto L e_t \hat{x}_{t-\tau}^\top$$

- For B :

$$\Delta \hat{B}_t \propto L e_t u_{t-\tau}^\top$$

- For L :

$$\Delta L_t \propto L e_t e_{t-\tau}^\top$$

- For C :

$$\Delta \hat{C}_t \propto e_t \hat{x}_{t+1-\tau}^\top$$

•

📌 Biologically plausible or not?

Here's the challenge:

- The initial gradients involve $C^\top$ (transpose of the observation matrix),
- This leads to **non-local learning rules** — not biologically realistic (neurons would need to know global matrix information).

🧠 **Their trick:** replace $C^\top$ with L , which is **locally available** (as input current into neurons representing $\hat{x}$).

This substitution is mathematically valid under certain assumptions and makes the updates **local and biologically plausible** — only dependent on the activity of nearby (pre/post-synaptic) neurons.

✅ 1) How is Bio-OFC only dependent on activity of nearby neurons, compared to the other approach?

The Bio-OFC network (green, panel A + B) is biologically plausible because:

🔄 All updates rely on local interactions:

- Neurons represent variables like $\hat{x}_t$, $e_t = y - C\hat{x}_t$, and $u_t = -K\hat{x}_t$.
- Synaptic weights encode matrices A, B, C, L, K .
- Updates to a weight (e.g. in matrix A) use **only**:
 - Presynaptic activity (e.g., $\hat{x}_{t-\tau}$),
 - Postsynaptic activity or input current (e.g., Le_t),
 - Which are **directly available at the synapse**.

There's no need to reference faraway neurons or global variables like the full matrix $C^\top$.

❌ **The alternative ANN (red, panel C) is not biologically plausible because:**

- It must **recompute all intermediate states** from $\hat{x}_{t+1-\tau}$ to $\tilde{x}_t$, which means:
 - You need **exact copies of A and B** at each step (unrealistic duplication).
 - You need to store a **long memory of past controls** $u_{t-\tau}, \dots, u_{t-1}$ — biologically hard.
- This creates **weight transport** and **temporal loop dependencies**, which violate locality.

Control: Policy Gradient

3.2.2 Control

Next, we consider optimal control a neural implementation of which was missing in most previous proposals, cf. Table 1. Traditionally, optimal control law is computed by iterating a matrix Riccati equation, cf. Supplementary Material, posing a difficult challenge for a biological neural implementation (but see [21]). To circumvent this problem, we propose to learn the controller weights $\mathbf{K}$ using a policy gradient method [12, 22] instead. Policy gradient methods directly parametrize a stochastic controller $\pi_{\mathbf{K}}(\mathbf{u}|\mathbf{x})$. Representing the total cost for a given trajectory τ as $c(\tau)$, they optimize the parameters $\mathbf{K}$ by performing gradient descent on the expected cost $J = \mathbb{E}_{\pi_{\mathbf{K}}} [c(\tau)] = \mathbb{E}_{\pi_{\mathbf{K}}} \left[\sum_{t=0}^T c_t \right]$.

$$\nabla_{\mathbf{K}} J = \int c(\tau) \nabla \pi_{\mathbf{K}}(\tau) d\tau = \mathbb{E}_{\pi_{\mathbf{K}}} [c(\tau) \nabla_{\mathbf{K}} \log \pi_{\mathbf{K}}(\tau)] \quad (17)$$

$$= \mathbb{E}_{\pi_{\mathbf{K}}} \left[\left(\sum_{t=0}^T c_t \right) \left(\sum_{s=0}^T \nabla_{\mathbf{K}} \log \pi_{\mathbf{K}}(\mathbf{u}_s | \mathbf{x}_s) \right) \right] \quad (18)$$

The term in square brackets is an unbiased estimator of the gradient and can be used to perform stochastic gradient decent. As already hinted at by [12], due to causality, costs c_t are not affected by later controls $\mathbf{u}_s$, $s > t$, and the variance of the estimator can be reduced by excluding those terms, which yields

$$\Delta \mathbf{K} \propto - \sum_{t=0}^T c_t \left(\sum_{s=0}^t \nabla_{\mathbf{K}} \log \pi_{\mathbf{K}}(\mathbf{u}_s | \mathbf{x}_s) \right). \quad (19)$$

We simply keep an eligibility trace of the past, $\mathbf{Z}_t = \sum_{s=0}^t \nabla_{\mathbf{K}} \log \pi_{\mathbf{K}}(\mathbf{u}_s | \mathbf{x}_s)$, and perform parameter updates $\Delta \mathbf{K} \propto -c_t \mathbf{Z}_t$ at each time step t . A similar update rule has been suggested by [23, 24] for the infinite horizon case. Global convergence of policy gradient methods for linear quadratic regulator has been recently studied by Fazel *et al.* [25].

We assume the output of neurons encoding control $\mathbf{u}$ is perturbed by Gaussian noise

$$\mathbf{u}_t = -\mathbf{K} \hat{\mathbf{x}}_t - \boldsymbol{\xi}_t \quad \text{with} \quad \boldsymbol{\xi}_t \sim \mathcal{N}(0, \sigma^2 \mathbf{I}). \quad (20)$$

The synapses are endowed with a synaptic tag [26] $\mathbf{Z}$, an eligibility trace that tracks correlations between pre-synaptic activity $\hat{\mathbf{x}}$ and post-synaptic noise $\boldsymbol{\xi}$. It is reset to zero at the beginning of each trajectory, though instead of a hard reset it could also softly decay with a time constant of the same order $\mathcal{O}(T)$ as trajectory duration [27]. The weight update assigns cost c_t to the synapses according to their eligibility $\mathbf{Z}_t$. The cost is e.g. provided by a diffuse neuromodulatory signal such as dopamine.

The optional use of momentum $m \in [0, 1)$ adds a low-pass filter to the synaptic plasticity cascade:

$$\mathbf{Z}_t = \mathbf{Z}_{t-1} + \boldsymbol{\xi}_t \hat{\mathbf{x}}_t^\top \quad \left(= \sigma^2 \sum_{s=0}^t \nabla_{\mathbf{K}} \log \pi_{\mathbf{K}}(\mathbf{u}_s | \hat{\mathbf{x}}_s) \right) \quad (21)$$

$$\mathbf{G}_t = m \mathbf{G}_{t-1} + c_t \mathbf{Z}_t \quad (22)$$

$$\Delta \mathbf{K}_t \propto -\mathbf{G}_t \quad (23)$$

What's the idea behind policy gradient?

You define a **stochastic controller**:

$$\pi_K(u|x)$$

Which gives a probability distribution over actions given a state. The goal is to minimize the **expected total cost**:

$$J = \mathbb{E}_{\pi_K}[c(\tau)] = \mathbb{E}_{\pi_K} \left[\sum_{t=0}^T c_t \right]$$

- You then take gradients of this cost with respect to the controller parameters K and **descend along the gradient**.

The network:

1. Tries slightly noisy actions.
2. Sees how well they worked (based on cost).
3. Adjusts control weights K using **local activity correlations**, guided by **global reward/punishment**.

- This replaces the mathematically heavy Riccati solution with a biologically inspired, lightweight learning rule.
-

🧠 How does the network do it?

Here's how it's implemented in the brain-like network:

🌐 Step 1: Add noise to the output control signal

$$u_t = -K\hat{x}_t - \xi_t, \quad \xi_t \sim \mathcal{N}(0, \sigma^2 I)$$

This randomness is **exploration** — it lets the network see the effect of slightly different actions.

🧠 Step 2: Track correlations between:

- The **noise** ξ_t ,
- The **presynaptic input** $\hat{x}_t$

This correlation is stored in an **eligibility trace** Z_t :

$$Z_t = Z_{t-1} + \xi_t \hat{x}_t^\top \quad (21)$$

This acts like a memory of how noise influenced actions over time.

💡 Step 3: Assign credit using the cost

The idea is: if the outcome (cost c_t) was bad, and noise ξ_t pushed the action a certain way, then maybe that direction was bad — so adjust K to avoid it.

Define:

$$G_t = mG_{t-1} + c_t Z_t \quad (22)$$

$$\Delta K_t \propto -G_t \quad (23)$$

This is a **local plasticity rule**: you update synaptic weights using:

- Pre-synaptic input ($\hat{x}_t$),
- Post-synaptic error signal (ξ_t),
- Global cost signal (c_t).

•

ssnOFC

SMD Implementation in the SCN paper

- Straight from the SCN repo with minor adjustments

Extend run_KfSCN_step with Online Learning

- Extended version of run_KfSCN_step that includes online learning of the Kalman filter gain Kf via local ($e_t * r_t^T$) updates.
- Two parts: Online K_f learning step and SCN voltage/spiking update
- 1) Online K_f learning step

```
# 1) Online K_f learning step
# Prediction error: e_t = y - C(D r)
e_t = y - C @ (D @ r)

# Local gradient: e_t (outer) r_t^T
# shape: (n,1)*(1,N)->(n,N) but K_f is (K,n). So we do transpose to match
# Kf is shape(K,n), e_t is shape(n,), r is (N,)
# => DeltaKf = etaKf * ( e_t r^T )^T = etaKf * (r e_t^T)
deltaKf = etaKf * np.outer(D @ r, e_t) # shape (K,n)

# Update Kf
Kf += deltaKf
```

Performing similar derivations for the other synaptic weights, our optimization procedure would rely on the following stochastic gradients:

$$-\nabla_A \frac{1}{2} e_t^T e_t = C^T e_t \hat{x}_{t-\tau}^T \quad -\nabla_B \frac{1}{2} e_t^T e_t = C^T e_t u_{t-\tau}^T \quad (11)$$

$$-\nabla_L \frac{1}{2} e_t^T e_t = C^T e_t e_{t-\tau}^T \quad -\nabla_C \frac{1}{2} e_t^T e_t = e_t \hat{x}_{t+1-\tau}^T \quad (12)$$

This yields a classical Hebbian rule between (a memory trace of) presynaptic activity $\hat{x}_{t+1-\tau}$ and postsynaptic activity e_t for weights C . However, it suggests non-local learning rules for A, B, L , which runs contrary to biological requirements. We can circumvent this problem by replacing C^T with L , which corresponds to left-multiplication of the gradients with a positive definite matrix (see Supplementary Material Sec. D). This still decreases the mean-square prediction error under some mild initialization constraints on C and L and yields local plasticity rules, cf. Fig. 1,

$$\Delta \hat{A}_t \propto L e_t \hat{x}_{t-\tau}^T \quad (13)$$

$$\Delta \hat{B}_t \propto L e_t u_{t-\tau}^T \quad (15)$$

$$\Delta L_t \propto L e_t e_{t-\tau}^T \quad (14)$$

$$\Delta \hat{C}_t \propto e_t \hat{x}_{t+1-\tau}^T \quad (16)$$

- BioOFC paper page 5, equation 16
- where we take $\hat{x} = D @ r$
- etaKf is the learning rate

```
# Recompute the SCN's Kalman connections after updating K_f:
O_k = - D.T @ Kf @ C @ D # shape (N,N)
F_k = D.T @ Kf # shape (N,n)
```


The optional use of momentum $m \in [0, 1)$ adds a low-pass filter to the synaptic plasticity cascade:

$$\mathbf{Z}_t = \mathbf{Z}_{t-1} + \xi_t \hat{\mathbf{x}}_t^\top \quad \left(= \sigma^2 \sum_{s=0}^t \nabla_{\mathbf{K}} \log \pi_{\mathbf{K}}(\mathbf{u}_s | \hat{\mathbf{x}}_s) \right) \quad (21)$$

$$\mathbf{G}_t = m \mathbf{G}_{t-1} + c_t \mathbf{Z}_t \quad (22)$$

$$\Delta \mathbf{K}_t \propto -\mathbf{G}_t \quad (23)$$

○

- BioOFC paper, page 6, equation 21 and 23
- skipped the optional momentum step on equation 22

```
# 7) Recompute SCN's control connectivity with updated Kc:
#   O_c = -D^T B Kc D
#   F_c = D^T B Kc
# (We assume B has the correct shape to multiply with Kc)
O_c = -D.T @ (B @ Kc) @ D
F_c = D.T @ (B @ Kc)

# 8) Normal SCN neuron updates: dv/dt
dvd_t = (
    -lam * v
    - O_f @ s
    + O_s @ r
    + (O_c @ r + F_c @ error_state) # control effect
    + (O_k @ r + F_k @ (C @ y))    # measurement update
)

# Euler step + voltage noise
v_next = v + dvd_t*dt + np.sqrt(dt)*sigma*np.random.randn(len(dvd_t))

# Threshold crossing
s_next = np.zeros_like(s)
above_thresh = np.where(v_next > T_vals)[0]
if len(above_thresh) > 0:
    # Let only the highest-voltage neuron spike
    winner = np.argmax(v_next)
    s_next[winner] = 1.0 / dt

# Update firing rates
drdt = s_next - lam * r
r_next = r + drdt * dt

• return r_next, s_next, v_next, Kc, Z, O_c, F_c, u_next
```

- from SCN repo directly

Experiments

- $\mathbf{K}_f$ is the Kalman filter gain matrix. It determines how much weight to assign to incoming (noisy and possibly delayed) sensory observations versus the internal model's predictions when updating the state estimate. In other words, it controls how the system fuses prior information with new measurements to produce an optimal estimate of the state.
- $\mathbf{K}_c$ is the control gain matrix associated with LQR control. It defines the feedback law that converts the state estimate into control signals. For example, in many linear quadratic regulators the control law is given by $\mathbf{u} = -\mathbf{K}_c \hat{\mathbf{x}}$, where $\hat{\mathbf{x}}$ is the estimated state. This

matrix is designed to minimize a cost function that penalizes both deviation from a target state and the magnitude of the control effort.

- **Measured Metrics:**
 - MSE (Estimation Accuracy): How close the decoded state is to the true system state.
 - Cost: Given that there is no control action (no control gain K_c and x_{ref} isn't provided), the cost here is purely from state deviations.
 - Spike Rate: The amount of neural spiking activity during the simulation.
 - This scenario tests the basic performance of the estimator when no online learning is applied.
- **Ex 1: Estimation Only with Fixed K_f**
 - In Scenario 1, the system's state is estimated by the SCN using a fixed Kalman gain K_f (precomputed offline by SCN's initialization function).
 - What's happening:
 - The simulation runs the SMD dynamics.
 - At every time step, the SCN processes the noisy observation y to update its internal state (firing rates r , spikes s , and voltages v) using the fixed connectivity (which includes the fixed K_f).
 - The network then decodes its state estimate as $\hat{x} = Dr$.
- **Ex 2: Estimation Only with Learned K_f**
 - In Scenario 2, everything is the same as in scenario 1 except that the network is now allowed to update K_f online using a learning rule.
 - What's happening:
 - The function `run_KfSCN_step_learn` is used instead of its fixed counterpart.
 - Here, at each time step, the network:
 - Decodes the current state estimate $\hat{x} = Dr$.
 - Compares it to the current observation y (via the mapping C) to generate an error e_t .
 - Uses the outer product (scaled by the learning rate) to adjust K_f
 - Recomputes the connectivity matrices (O_k and F_k) based on the updated K_f
- **Ex 3: Control Only with Fixed K_f**
 - In Scenario 3, the network now includes a control component, but the estimation part (Kalman gain K_f) remains fixed. Only the control gain K_c is learned online via a policy gradient update.
 - What's happening:
 - The simulation continues as before for the state estimation (with fixed K_f), so the network estimates the state using the same fixed parameters. Now, with a

reference target state x_{ref} provided, the network uses its estimate to compute a control action according to:

$$\circ \quad u = -K_c(\hat{x} - x_{\text{ref}}) + \text{noise}$$

- The function `run_SCNcontrol_step_learn` implements this with an additional local update rule for K_c
 - It adds exploration noise.
 - It evaluates an instantaneous cost (combining state deviation and control effort).
 - It uses an eligibility trace (accumulating correlations between the exploration noise and the state estimate) to compute a policy gradient update ΔK_c so that K_c is updated in the direction that minimizes cost.
 - The updated control connectivity is then recomputed.
- **Ex 4: Full System - Learned K_f and K_c**
 - In Scenario 4, both the estimation gain K_f and the control gain K_c are adapted online.
 - What's happening:
 - The network uses `run_KfSCN_step_learn` to update K_f as in Scenario 2.
 - Simultaneously, it uses `run_SCNcontrol_step_learn` to update K_c as in Scenario 3.
 - As a result, the system's internal state estimator and the control policy both adapt based on the local errors.

Closed-form control with spike coding networks

Slijkhuis

x, s, r

- $x(t)$

The “true” signal or state you want the network to represent or control.

- In motor control it might be hand position, angle of a pendulum, velocity of a car, etc.
- Mathematically it lives in $\mathbb{R}^M$.

- $s(t)$

The **raw spikes** emitted by the neurons.

- For an N -neuron network, $s(t)$ is an N -vector of Dirac delta pulses:

$$s_i(t) = \sum_k \delta(t - t_i^k),$$

where t_i^k are the times neuron i fires.

- Conceptually: “when you see a spike in neuron i , set s_i to an impulse.”

- $r(t)$

A **smoothed version** of $s(t)$, i.e. the network’s internal “firing-rate” trace.

- Defined by

$$\dot{r}_i = -\lambda_d r_i + \lambda_d s_i.$$

- Intuition: each spike in s_i jumps r_i up by λ_d , then r_i decays back down with rate λ_d .
- So $r_i(t)$ tracks “how often neuron i has been spiking recently”—the smoother signal you use to **reconstruct x** .

●

—

Recurrent Leaky Integrate-and-Fire (LIF) Neural Network

The spiking patterns are generated according to some underlying voltage dynamics. There are many such models of varying complexities, but for most practical applications networks of leaky integrate-and-fire (LIF) neurons suffice. A network of N such neurons is then defined by

$$\dot{\mathbf{v}}(t) = -\lambda \mathbf{v}(t) + \mathbf{F} \mathbf{c}(t) + \mathbf{\Omega}_s \mathbf{r}(t) + \mathbf{\Omega}_f \mathbf{s}(t) + \boldsymbol{\eta}_v, \quad (1)$$

where $\mathbf{v}(t) \in \mathbb{R}^N$ is the vector of neural voltages, $\mathbf{F} \in \mathbb{R}^{K \times N}$ are some input weights, $\mathbf{c}(t) \in \mathbb{R}^K$ is some K -dimensional input, $\mathbf{s}(t) \in \mathbb{R}^N$ are the emitted spikes, $\mathbf{r}(t) \in \mathbb{R}^N$ are the filtered spike-trains¹, and $\boldsymbol{\eta}_v \in \mathbb{R}^N$ corresponds to independent voltage noise. Finally, there are fast synapses which affect the post-synaptic voltage instantaneously following a spike (through fast recurrent connections $\mathbf{\Omega}_f \in \mathbb{R}^{N \times N}$), and slow synapses which cause an initially slow change in voltage (through the slow recurrent connections $\mathbf{\Omega}_s \in \mathbb{R}^{N \times N}$).

Whenever a given neuron's voltage v_i reaches a threshold T_i , that neuron will emit a spike at that time (resulting in spike-trains $s_i(t) = \sum_j \delta(t - t_j)$). The voltage will then be reset to some resting value (here implemented through the diagonal elements of $\mathbf{\Omega}$).

$$\dot{\mathbf{v}}(t) = -\lambda \mathbf{v}(t) + F_c \mathbf{c}(t) + \mathbf{\Omega}_s \mathbf{r}(t) + \mathbf{\Omega}_f \mathbf{s}(t) + \boldsymbol{\eta}_v(t)$$

Closed-form control wit...

Term	Math definition	Conceptual role
$\dot{\mathbf{v}}(t)$	$\frac{d}{dt} v_i(t)$ for each neuron $i = 1 \dots N$.	Instantaneous rate of change of each neuron's membrane potential.
Leak $-\lambda \mathbf{v}(t)$	$\lambda > 0$ is a scalar; yields $\dot{v}_i \leftarrow -\lambda v_i$.	Passive decay of voltage back toward rest (0), with time constant $1/\lambda$.
Feed-forward $F_c \mathbf{c}(t)$	$\mathbf{c}(t) \in \mathbb{R}^M$ is the external input (e.g. concatenated sensory $y(t)$ and reference $z(t)$); $F_c \in \mathbb{R}^{N \times M}$.	Drives each neuron with current that encodes both what the network senses and the target it should track.
Slow recurrent $\mathbf{\Omega}_s \mathbf{r}(t)$	$\mathbf{r}(t)$ is the vector of low-pass-filtered spike trains ($\dot{r} = -\lambda_d r + \lambda_d s$); $\mathbf{\Omega}_s \in \mathbb{R}^{N \times N}$.	Implements the continuous dynamics of the system (or estimator, or controller) by feeding back past activity through "slow" synapses.
Fast recurrent $\mathbf{\Omega}_f \mathbf{s}(t)$	$\mathbf{s}(t)$ is the raw spike train (sum of Dirac deltas); $\mathbf{\Omega}_f \in \mathbb{R}^{N \times N}$.	Instantaneous feedback that balances excitation and inhibition and enforces the greedy spike reset rule.
Noise $\boldsymbol{\eta}_v(t)$	A vector of independent Gaussian white-noise processes (one per neuron).	Captures random membrane fluctuations and ensures realistic variability.

1. **Leak** ($-\lambda v$) makes sure voltages don't drift off to infinity—neurons “forget” in the absence of input.
2. **Feed-forward drive** $F_c c(t)$ injects **new information** every millisecond about what the network is supposed to estimate or control.
3. **Slow recurrent currents** $\Omega_s r$ implement the **ideal continuous-time flow** of the underlying dynamical system (or Kalman update or LQR law)—they push the network's internal state along the trajectories it should follow.
4. **Fast recurrent currents** $\Omega_f s$ are instantaneous kicks whenever any neuron spikes, creating a tight **excitation–inhibition balance** and resetting the spiking neuron below threshold.
5. **Noise** η_v adds small random perturbations, which in practice helps keep the network from locking into pathological firing patterns.

By choosing

$$\Omega_s = D^\top (A + \lambda I) D, \quad \Omega_f = -D^\top D,$$

and packing all external inputs into F_c , this single LIF recurrence faithfully reproduces the **linear-dynamics** term and the **autoencoder** condition from Boerlin et al. (2013), but now extended in Slijkhuis et al.

- to also include estimation and control pathways Closed-form control with....
- a spike occurs when v_i crosses threshold T_i , resetting via the diagonal of Ω_f .
- This setup poses the question: **How do we set $\{F_c, \Omega_s, \Omega_f, D\}$ to achieve optimal LQG control?**

Spike-Coding Network Theory

- **Greedy Spiking Rule**

spike *iff* $\|\mathbf{x} - \mathbf{D}\mathbf{r}\|_2^2 > \|\mathbf{x} - \mathbf{D}\mathbf{r} - \mathbf{D}_i\|_2^2$. Here $\mathbf{D}_i$ is the i 'th column of $\mathbf{D}$, and reflects the change in error due to neuron i spiking.

- **Interpretation**

- If adding $\mathbf{D}_i$ makes the error $\|\mathbf{x} - \hat{\mathbf{x}}\|$ *smaller*, you fire; otherwise you stay silent.
- This is a purely *greedy* rule, local to each neuron: it only needs to know the current error vector $\mathbf{x} - \mathbf{D}\mathbf{r}$ and its own decoder $\mathbf{D}_i$.

-